

高级语言程序设计

实验报告

南开大学 计算机科学与技术（伯苓班）

姓名 罗熙媛

学号 2514039

班级 计算机伯苓二班

2026年5月14日

高级语言程序设计大作业实验报告

一. 作业题目

《南开大学循环校园》—— 交互叙事文字游戏

二. 开发软件

IDE: Qt Creator 12.0 + Visual Studio 2022

框架: Qt 6.6 (Widgets 模块)

编译器: MSVC 2022 (64-bit)

三. 课题要求

- 1) 图形化界面: 使用 C++ Qt 框架实现窗口、按钮、文本浏览器等 GUI 组件。
- 2) 面向对象: 采用 GameState (状态类)、SceneEngine (场景引擎类)、IGameView (视图接口) 等类层次结构。
- 3) 状态持久化: 实现多槽位存档/读档功能, 支持跨循环保留关键数据。
- 4) 多媒体支持: 背景音乐播放 (MP3)、界面背景图片显示。
- 5) 交互叙事: 分支剧情、概率事件、结局系统。

四. 主要流程

1. 整体流程

游戏采用状态机 + 事件驱动架构，核心流程如下：

- 1) 开场动画：玩家经历第一次循环，发现作业丢失。
- 2) 循环选择：玩家可选择“继续做作业”进入 B1 结局路径，或“探索真相”解锁全部校园场景。
- 3) 校园探索：通过 HUD 上的场景按钮进入宿舍、食堂、公教楼、图书馆、操场、到处看看等子场景。
- 4) 时间推进：每次行动后时段自动推进（早晨→上午→中午→下午→傍晚→夜晚），夜晚需选择是否回宿舍，否则扣减体力与精神。
- 5) 任务与结局：完成三个主线任务后解锁 T1/T2 结局入口；集齐三个世界裂缝并解锁 T1+T2 后可进入隐藏结局。
- 6) 循环重置：周日夜晚入睡后，循环计数器 L 加一，部分状态重置（如当日临时标志），但知识、身体素质、已完成任务、已解锁结局等永久保留。

2. 算法或公式

(1) 游戏状态机

GameState 类集中存储所有可变数据：

```
int E;      // 体力 0-100

int S;      // 精神稳定 0-100

int K;      // 知识

int Q;      // 身体素质

int L;      // 循环次数
```

```

int M;          // 沉沦值（触发 B2 结局）

QSet<QString> items;          // 当前持有道具

QSet<QString> completedTasks; // 已完成任务（跨循环）

QSet<QString> unlockedEndings; // 已解锁结局入口

bool canteenAuntUnlocked;     // 食堂阿姨是否解锁（持久）

bool catTeachingSat;          // 公教楼陪猫（跨循环累计）

```

(2) 场景调度算法

每个场景对应一个成员函数（如 `dormScene()`、`canteenScene()`），场景内调用 `showChoices()` 展示选项。选择后执行相应逻辑，最后调用 `postDaytimeAction()` 推进时间：

```

void SceneEngine::postDaytimeAction(std::function<void()> next) {

    if (slot == Night) {

        // 夜晚直接结算并进入下一天早晨

        enterNextDayMorning();

        return;

    }

    advanceTime();      // 时段 +1

    refreshHud();

    if (slot == Night) {

        askReturnToDormIfNight(next);

    } else if (next) next();

}

```

(3) 时间推进与循环重置

1) 时段序列: Morning → Forenoon → Noon → Afternoon → Evening
→ Night。

2) 跨天逻辑: 若当前为周日夜晚, 则调用 endLoopAndRespawn(): L++,
重置所有 *ThisLoop 临时标志, 但保留 K、Q、已完成任务等。

3) 持久标志 (如 catTeachingSat、everCompletedHandkerchief) 不在
resetLoopFlags() 中清空, 从而实现跨循环积累。

(4) 随机事件系统

采用概率判定 + 权重选择, 让每次探索结果不同:

```
static bool chance(int percent) {  
    return (rand() % 100) < percent;  
}  
  
static int pickWeighted(const QList<int>& weights) {  
    int total = std::accumulate(weights.begin(), weights.end(), 0);  
    int r = rand() % total;  
    int acc = 0;  
    for (int i = 0; i < weights.size(); ++i) {  
        acc += weights[i];  
        if (r < acc) return i;  
    }  
    return weights.size() - 1;  
}
```

(5) 存档/读档序列化

使用 Qt 的 `QJsonObject` 将所有 `GameState` 成员变量(包括 `QSet` 和布尔标志) 转换为 JSON, 写入本地文件(支持 5 个槽位)。同时保存当前的对话快照 (`lastDialogText`、`lastChoiceLabels`) 以便读档后恢复上下文。示例:

```
QJsonObject root;

root["E"] = m_state->E;

root["items"] = setToArray(m_state->items);

// ... 写入所有字段

QFile f(slotPath(slot));

f.write(QJsonDocument(root).toJson());
```

3. 单元测试

对关键逻辑编写了测试用例(在独立控制台项目中验证):

测试模块	测试内容	预期结果
<code>chance(30)</code> 概率	调用 10000 次, 统计成功次数	约 3000 ± 200
<code>pickWeighted</code> 权重	权重 {30, 50, 20}, 测试 10000 次	各区间比例近似 30%/50%/20%
状态机属性修改	体力值加 15 后 <code>clamp</code> 到 100	E 不超过 100
循环重置标志	重置后 <code>tryProbeRoommateDone</code> 变为 <code>false</code>	标志正确清空
持久标志保留	重置后 <code>canteenAuntUnlocked</code> 仍为 <code>true</code>	不丢失

所有测试通过。

五. 单元测试

测试案例表

测试场景	输入/操作	期望输出	结果
体力值边界	E=95, 使用“速溶咖啡”(+15)	E=100	✓
精神值归零	S=3, 执行“再睡一会儿”(-5)	S=-2, 触发文字乱码	✓
食堂阿姨解锁	L=7 时概率 30% 触发 apron	<code>canteenAuntUnlocked=true</code>	✓
道具使用移除	背包有“梨膏糖”, 点击使用	道具消失, S 增加	✓
存档保存/加载	保存当前状态, 关闭程序后读取	所有数值与标志一致	✓
周日晚上睡眠循环	L=2, 任务未完成, 入睡	L=3, 回到周六早晨	✓
隐藏结局条件	三个裂缝 + T1/T2 解锁, 周日夜去马蹄湖	弹出两个分支选项	✓

所有功能正常运行, 随机事件分布符合设计预期。

六. 收获

1. 虚析构函数与多态

GameState 作为数据容器没有继承关系, 但 SceneEngine 和 MainWindow 之间通过 IGameView 接口交互。在 MainWindow 析构时, 需要正确释放 SceneEngine 指针 (std::unique_ptr 自动管理), 避免了手动 delete 可能引发的资源泄漏。

2. Qt 资源文件的播放问题

背景音乐 bgm.mp3 被编译到 Qt 资源系统 (qrc) 中, 但 Windows MCI 接口无法直接识别 ./bgm.mp3 路径。解决办法: 在 playBgm() 中检测路径前缀, 若为 ./ 或 qrc:, 则先用 QFile 读出全部数据, 写入 QDir::tempPath() 下的临时文件, 再将临时文件的本地路径传给 mciSendString。这样既保持了资源内嵌的便利性, 又兼容了多媒体 API。

3 . 跨循环状态保持

设计初期将全部标志放在同一个结构体中，每次循环重置时容易误清持久标志。

后来将标志分为两类：

1) 临时标志（tryProbeRoommateDone、teachingAa 等）：在 resetLoopFlags() 中清零。

2) 持久标志（canteenAuntUnlocked、catTeachingSat、everCompletedHandkerchief 等）：由开发者手动维护，不在重置函数中处理。

这种分类保证了玩家在多次循环中逐步积累进度，符合叙事设计。

此外，学习使用 QSet<QString> 存储道具和完成任务，利用集合运算快速判断 hasItem()，简化了道具系统的实现。

七. AI 工具使用披露

本项目的开发过程中，在以下环节使用了生成式 AI 工具辅助完成：

AI 工具名称及版本	具体用途	使用位置
DeepSeek (2025 年 5 月版)	1. 代码片段生成与优化（如随机事件权重函数、存档 JSON 序列化）	代码：scenes.cpp 中的 pickWeighted、GameState 序列化逻辑；
	2. 剧情文本润色与结局文案辅助撰写	剧情：结局 T1、T2、隐藏结局的部分修辞润色；
	3. 调试过程中解释错误原因（如 MCI 无法播放资源文件的解决方案）	报告：第四、六节的初稿框架。
	4. 实验报告结构与部分技术描述草稿	

说明：所有 AI 生成的代码和文本均经过本人审阅、修改和测试，符合项目设计意图。未直接复制未经检查的 AI 输出。AI 工具主要提升了开发效率，核心算法、游戏机制设计、场景逻辑与结局构思均由本人独立完成。